1.

a) Compare AWT and Swing with respect to components and design.

AWT
• Uses native OS components
• Platform dependent
• Heavyweight components
• Limited controls
• Less flexible UI
Swing
• Pure Java components
• Platform independent
• Lightweight components
• Rich set of controls
• Supports pluggable look and feel
Key difference
• AWT depends on OS, Swing does not

b) Create a Swing application using JFrame and JButton.

```
import javax.swing.*;
public class SwingDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("Swing Example");
        JButton b = new JButton("Click Me");
        b.setBounds(100, 100, 120, 40);
        f.add(b);
        f.setSize(300, 300);
        f.setLayout(null);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

c) Analyze why Swing is preferred for modern GUI applications.

• Platform independent
• Rich UI components
• Better customization
• Supports MVC architecture
• Improved user experience

2.

a) Explain the working of JLabel, JTextField, JCheckBox, and JRadioButton.

JLabel
• Displays text or image
• Non-editable

JTextField
• Accepts single-line input
• Editable by user

JCheckBox
• Allows multiple selections
• Independent options

JRadioButton
• Allows single selection
• Used with ButtonGroup

b) Design a Swing form implementing these controls.

```
import javax.swing.*;
public class FormDemo {
   public static void main(String[] args) {
      JFrame f = new JFrame("Registration Form");
      JLabel l1 = new JLabel("Name:");
      l1.setBounds(50, 50, 100, 30);
      JTextField t1 = new JTextField();
      t1.setBounds(150, 50, 150, 30);
      JCheckBox c1 = new JCheckBox("Java");
      c1.setBounds(50, 100, 100, 30);
      JCheckBox c2 = new JCheckBox("Python");
      c2.setBounds(150, 100, 100, 30);
      JRadioButton r1 = new JRadioButton("Male");
      JRadioButton r2 = new JRadioButton("Female");
      r1.setBounds(50, 150, 100, 30);
      r2.setBounds(150, 150, 100, 30);
      ButtonGroup bg = new ButtonGroup();
      bg.add(r1);
      bg.add(r2);
      JButton b = new JButton("Submit");
      b.setBounds(100, 200, 100, 30);
      f.add(l1); f.add(t1);
      f.add(c1); f.add(c2);
      f.add(r1); f.add(r2);
      f.add(b);
      f.setSize(400, 300);
      f.setLayout(null);
      f.setVisible(true);
      f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
   }
}
```

c) Evaluate how control selection impacts usability and data correctness.

• Proper controls reduce user errors
• Radio buttons prevent invalid multiple selection

• Checkboxes allow flexibility
• Text fields ensure structured input
• Improves accuracy and usability

3.

a) Explain the purpose of JTabbedPane, JList, and JComboBox.

JTabbedPane
• Organizes content into tabs
• Improves navigation

JList
• Displays list of items
• Allows selection (single/multiple)

JComboBox
• Dropdown list
• Saves space
• Allows single selection

b) Develop a program integrating these components into a single interface.

```
import javax.swing.*;
public class AdvancedSwing {
    public static void main(String[] args) {
        JFrame f = new JFrame("Dashboard");
        JTabbedPane tp = new JTabbedPane();
        JPanel p1 = new JPanel();
        JPanel p2 = new JPanel();
        String data[] = {"Item1", "Item2", "Item3"};
        JList list = new JList(data);
        String options[] = {"Option1", "Option2", "Option3"};
        JComboBox cb = new JComboBox(options);
        p1.add(list);
        p2.add(cb);
        tp.add("List Tab", p1);
        tp.add("Combo Tab", p2);
        f.add(tp);
        f.setSize(300, 300);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

c) Analyze how such components improve navigation and user experience.

• Tabs organize content clearly
• Lists allow quick selection

• ComboBox saves space
• Improves navigation
• Enhances user experience